ECE 5755 Lab 1: Kernel Implementation and Top-Down Analysis

Intel Xeon Silver 4514Y (Sapphire Rapids); Level-2 profiling, size-500 runs

Name: Abiola **NetID:** ab3394

1. Kernel implementations

Convolution

The `convolution()` function computes a valid multi-channel 2-D convolution with unit stride and no padding. Its output is stored as a three-level pointer structure indexed by filter, output row, and output column. Each output element starts at its filter bias, and nested loops then accumulate the product of the corresponding input pixel and kernel coefficient over every input channel and kernel row and column. For input side length N and kernel side length K, the output side length is $N - K + 1$, so the arithmetic work is $O(F(N - K + 1)^2 CK^2)$, where F is the number of filters and C is the number of channels. The test driver initializes a 3-channel image and 2 filters.

ReLU

The scalar `relu()` function returns its input unchanged when $x > 0$ and zero for zero or negative values. `applyRelu()` applies this operation in place to every element of an input array, avoiding a second output allocation and giving linear work in the input size. The functional test checks positive, zero, and negative examples.

Linear layer

The `linear()` function implements a fully connected layer. It allocates an output vector with one value per output neuron, initializes each value with the matching bias, and accumulates the dot product between the input vector and that neuron's weight row, with weights addressed as `weights[outputIndex][inputIndex]`. Its work is $O(IO)$ for input size I and output size O. The caller owns and frees the returned vector.

Matrix multiplication

The `matmul()` function returns `NULL` when the left matrix column count does not equal the right matrix row count (`A_cols != B_rows`). For compatible inputs, it allocates a row-pointer result matrix and uses the conventional three-loop algorithm over result row, result column, and inner product: each `C[i][j]` starts at zero and accumulates `A[i][k] * B[k][j]` over the shared dimension. Its work is $O(A_{rows} A_{cols} B_{cols})$. The caller frees the result rows and outer pointer.

Softmax

The `softmax()` function returns log-probabilities. It finds the maximum input value and subtracts it before calling `exp()`, which reduces overflow risk without changing the normalized distribution. It stores the exponentials in a temporary array, sums them, writes `log(expValue / sum)` to a separate output vector, and frees the temporary array. Exponentiating the outputs gives probabilities whose sum is approximately one. The caller owns and frees the returned log-probability vector.

2. Tests implemented

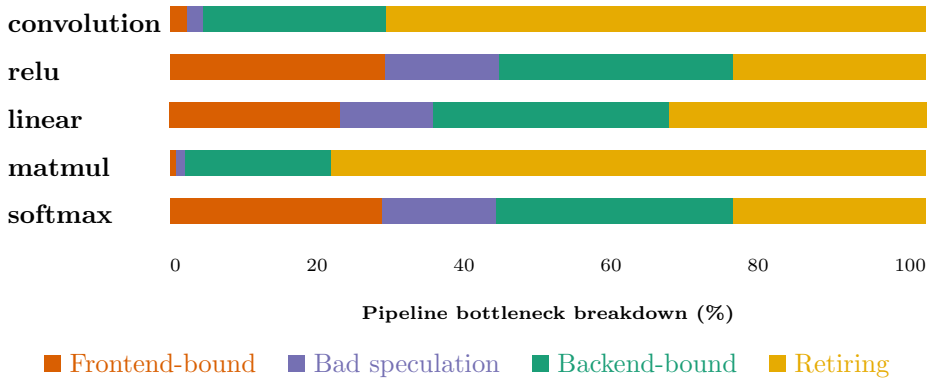
The Unity test driver in `tests/all_tests.c` runs tests for convolution, neural-network helpers, functional operations, linear layers, and matrix operations. The tests check both ordinary cases and boundary or error cases: convolution behavior, flattening and prediction, softmax normalization, ReLU behavior, linear layers with zero/negative/large inputs and parameters, square matrix multiplication, and incompatible matrix dimensions.

The purpose of these tests is to verify numerical correctness independently of performance measurements. In particular, dimension checks and negative or zero values expose indexing, initialization, bias, and branch-handling errors that may not appear in one ordinary example.

Additional test rationale: The test driver runs the linear cases for zero weights and biases, negative weights and biases, and large-magnitude inputs, weights, and biases. Neural-network tests check channel-major flattening and prediction for a maximum, a tie, and mixed signs. Softmax is checked by verifying that exponentiated outputs sum to one and preserve the maximum index; matrix multiplication is checked for both a known product and incompatible dimensions. Some linear test functions are registered more than once in the driver, so the total invocation count includes repeats.

3. Top-down profiling results

I profiled each standalone kernel driver with `pmu-tools/toplev.py` on an instructional compute node. The command used Level 2, the Sapphire Rapids CPU override, and a fixed input size of 500. Each bar below shows the Level-1 top-down categories and totals 100%: frontend bound, bad speculation, backend bound, and retiring.

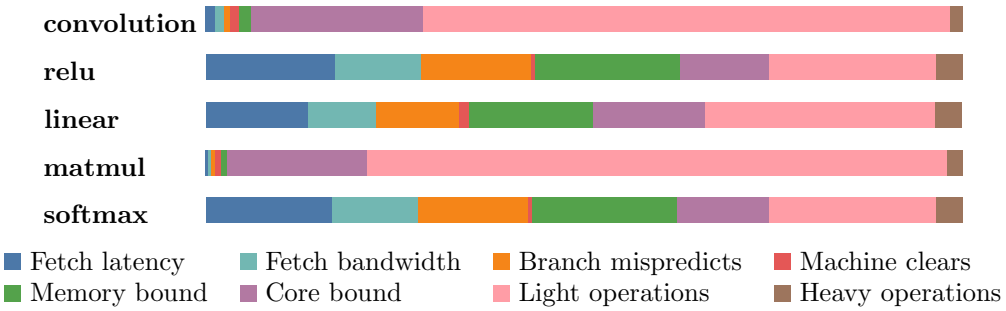


Kernel	Frontend	Bad spec.	Backend	Retiring
convolution()	2.3%	2.0%	24.3%	71.4%
relu()	28.4%	15.1%	31.0%	25.5%
linear()	22.5%	12.3%	31.3%	34.0%
matmul()	0.8%	1.2%	19.3%	78.7%
softmax()	28.0%	15.1%	31.4%	25.5%

Profiling procedure. I ran one measurement per kernel for the chart at size 500 on the instructional Sapphire Rapids compute node, using `LEVEL=2`. The runner invokes `make <kernel> LEVEL=2 SIZE=500`; the Makefile builds the selected driver and invokes `toplev.py --core S0-C0 -l2 -v --no-desc --force-cpu spr ./<kernel> 500`. The runner redirects each command's standard output and error to its own file in `perf_results/`. All five size-500 files report exit status 0 and `MUX=100%`. These are single-run measurements, not averages.

Level-2 breakdown

The following chart expands each Level-1 result into the Level-2 categories reported by `toplev`. Values are from the size-500 runs; each bar is normalized to 100% and follows the order shown in the legend.



Convolution observations and bottleneck

Your interpretation: The convolution implementation required a substantial amount of arithmetic work, which is reflected in the high retiring percentage. The main secondary limitation was execution-resource pressure, as indicated by the core-bound fraction, rather than delays due to memory fetches.

ReLU observations and bottleneck

Your interpretation: The ReLU implementation involves a conditional branch for each element, which can lead to branch mispredictions depending on the input distribution. The relatively balanced profile indicates that memory access and simple arithmetic are not the primary bottlenecks, and the small size of the operation allows a significant portion of instructions to retire efficiently.

Linear observations and bottleneck

Your interpretation: The linear layer's main bottleneck appears to be memory-bound, as indicated by the 16.4% memory bound fraction. This suggests that the weight layout and cache behavior significantly impact performance, and optimizing the loop order to improve data locality could help reduce memory stalls.

Matrix multiplication observations and bottleneck

Your interpretation: The matrix multiplication's high retiring fraction indicates that most of the computation is useful arithmetic work. The naive loop order may not fully exploit cache locality, but the predictable access patterns and regular computation allow the core to remain busy, minimizing frontend and speculation stalls. Optimizing the loop order for better cache reuse could further improve performance.

Softmax observations and bottleneck

Your interpretation: The softmax implementation involves multiple passes over the input data, including computing the exponentials, summing them, and normalizing. This leads to temporary array usage and additional arithmetic operations. The profile resembles ReLU in terms of frontend and speculation fractions due to the conditional and loop overhead, but the backend bound is slightly higher because of the extra memory accesses and arithmetic involved in the multiple passes.

4. Debugging process

I used the failing unit tests to isolate incorrect outputs, then ran the test binary under Valgrind with origin tracking. Valgrind reported an invalid read in `flatten()`; the code indexed the image as row-column-channel even though the allocated image is channel-row-column. Changing the access to `input[channel][row][column]` fixed the out-of-bounds read and stopped it from corrupting later tests. The softmax test also exposed that the function returned normalized probabilities while the implementation contract is log-softmax; returning the logarithm of each normalized exponential fixed the check that exponentiated outputs sum to one. After these corrections, the test driver reported 22 passing invocations and no failures. I also used profiler output to identify environment issues: perf counters were denied on the login node, and pinning with `taskset -c 0` failed when CPU 0 was outside the Slurm CPU set. Profiling succeeded after running on a compute node and removing the fixed CPU pin.

5. Performance improvements

The top-down data suggests several optimization directions. Frontend-bound kernels, such as ReLU and softmax, could benefit from reducing instruction and branch overhead, simplifying loops, and enabling compiler optimization or vectorization. Backend-bound kernels, such as the linear layer and convolution, could benefit from better data locality, contiguous allocations, improved loop order, blocking or tiling, and fewer temporary allocations, which reduce memory stalls and repeated memory accesses. More specifically, ReLU may benefit from branchless or vectorized execution; linear and convolution from contiguous layouts and SIMD-friendly inner loops; matrix multiplication from cache blocking; and softmax from fewer passes and less temporary storage while preserving numerical stability.

6. Profiling issues and reproducibility

During profiling, I encountered several issues, including small-input overhead, counter access restrictions, and Slurm time limits. I ran the measurements on an instructional compute node and saved the outputs in the `perf_results/` directory for reproducibility.

Source disclosure. GitHub Copilot was used for coding assistance. No other external code or documentation was used.